Experiment Reporting

LLM names, versions, dates and parameters

For our study we used two different LLMs: one for generating questions based on the code written by participants, and another one for translating questions from English to a different language (Spanish or Italian). In what follows we report the details of each LLM and their usage, including provider, name, version, dates in which the LLMs were used, and parameters employed when making the LLM (API) calls.

Model used for generating questions

- Provider: OpenAI.
- Name: o1.
- Version: `o1-2024-12-17`.
- Dates: March - June 2025.
- Parameters: All parameters used were the default ones as documented in https://platform.openai.com/docs/api-reference/chat/create except for the prompts, documented in the next section.

Model used for translating questions

- Provider: OpenAI.
- Name: GPT-4o mini.
- Version: `gpt-4o-mini-2024-07-18`.
- Dates: March - June 2025.
- Parameters: All parameters used were the default ones as documented in https://platform.openai.com/docs/api-reference/chat/create except for the prompts, documented in the next section.

Prompts used

We used a total of seven different prompts in our pipeline, six of them for generating questions and the remaining one for translating questions. We used six prompts for generating questions because we had to accommodate two different tasks (development and maintenance) and three different scenarios (default, Master students, and Italian students). In what follows we report the prompts used. Content within double curly braces (`{{}}`) are placeholders dynamically replaced at inference time.

Prompt for generating questions for development task

```
You are a lecturer in Computer Science whose goal is to evaluate the source
code of a program written by a student to implement a specific request
described below (# Task).
```

```
You must carefully analyze the student's code and ask questions to find out
if they understand the code they have written.
You must ask six questions whose answer must be a line of code related to
the question. For instance, you may ask where in the code a certain
condition is checked, where certain functionality is accomplished, or where
certain behavior is handled, among other options.
Avoid providing explicit details in the questions such as literal strings
or conditions, since then those would be easy to spot in the code, without
actually requiring an understanding of it. Avoid making reference to
previous questions you wrote since the questions will be asked in a random
order.
You must also ask three open-ended questions, where the student must write
a textual answer. Ask questions about the implemented logic, to verify that
they understood what they implemented.
One of these three questions must have the form: "Describe in natural
language all code blocks in the <function_name> function". Pick as
<function_name> the name of the function you consider most complex among
the ones implemented by the student.
Remember, your goal is not to determine whether the code is correct or not,
but to find out whether the student has understood what they have done. The
student might have copied the code from some external source such as Stack
Overflow, or they might have used tools such as GitHub Copilot or ChatGPT,
so it is not certain that all the code was written by them and, therefore,
it is not certain that they understand it.

The code for which you should pose the questions is as follows:

{{code}}

# Task

{{task}}

Next, write the nine questions. In order to automate the assessment of the
responses provided by the student, write also the answer to the questions
asking for a line of code. Write the questions and answers and nothing
else. Use the following template to write the questions and answers:

1. Copy and paste a line or lines of code: <question>. ANSWER TO THE
QUESTION: `<answer>`.

2. Copy and paste a line or lines of code: <question>. ANSWER TO THE
QUESTION: `<answer>`.

3. Copy and paste a line or lines of code: <question>. ANSWER TO THE
QUESTION: `<answer>`.

4. Copy and paste a line or lines of code: <question>. ANSWER TO THE
QUESTION: `<answer>`.

5. Copy and paste a line or lines of code: <question>. ANSWER TO THE
QUESTION: `<answer>`.

6. Copy and paste a line or lines of code: <question>. ANSWER TO THE
QUESTION: `<answer>`.

7. Provide a textual answer: Describe in natural language all code blocks
in the <function_name> function.

8. Provide a textual answer: <question>.
```

```
9. Provide a textual answer: <question>.
```

Prompt for generating questions for maintenance task

```
You are a lecturer in Computer Science whose goal is to evaluate the source code of a program modified by a student to implement a specific request described below (# Task).
You must carefully analyze the original code as well as the differences between the code submitted by the student and the original code, and ask questions to find out if they understand the new code they have written.
You must ask six questions whose answer must be a line of code related to the question. For instance, you may ask where in the code a certain condition is checked, where certain functionality is accomplished, or where certain behavior is handled, among other options.
Avoid providing explicit details in the questions such as literal strings or conditions, since then those would be easy to spot in the code, without actually requiring an understanding of it. Avoid making reference to previous questions you wrote since the questions will be asked in a random order.
You must also ask three open-ended questions, where the student must write a textual answer. Ask questions about the implemented logic, to verify that they understood what they implemented.
One of these three questions must have the form: "Describe in natural language all code blocks in the <function_name> function". Pick as <function_name> the name of the function you consider most complex among the ones implemented by the student.
Your questions must be related only to the new code written by the student, not to the original code.
Remember, your goal is not to determine whether the code is correct or not, but to find out whether the student has understood what they have done.
The student might have copied the code from some external source such as Stack Overflow, or they might have used tools such as GitHub Copilot or ChatGPT, so it is not certain that all the code was written by them and, therefore, it is not certain that they understand it.

The original code of the program is as follows:

{{code}}

And the diff between the code written by the student and the original code is as follows:

```
{{diff}}
```

Task

{{task}}

Next, write the nine questions. In order to automate the assessment of the responses provided by the student, write also the answer to the questions asking for a line of code. Write the questions and answers and nothing else. Use the following template to write the questions and answers:

1. Copy and paste a line or lines of code: <question>. ANSWER TO THE QUESTION: `<answer>`.
```

```
2. Copy and paste a line or lines of code: <question>. ANSWER TO THE
QUESTION: `<answer>`.

3. Copy and paste a line or lines of code: <question>. ANSWER TO THE
QUESTION: `<answer>`.

4. Copy and paste a line or lines of code: <question>. ANSWER TO THE
QUESTION: `<answer>`.

5. Copy and paste a line or lines of code: <question>. ANSWER TO THE
QUESTION: `<answer>`.

6. Copy and paste a line or lines of code: <question>. ANSWER TO THE
QUESTION: `<answer>`.

7. Provide a textual answer: Describe in natural language all code blocks
in the <function_name> function.

8. Provide a textual answer: <question>.

9. Provide a textual answer: <question>.
```

Prompt for generating questions for development task for Master students

```
You are a lecturer in Software Testing whose goal is to evaluate the code
of a program and test suite written by a student to implement a specific
request described below (# Task).
You must carefully analyze the student's code and ask tough questions to
find out if they understand the code they have written.
You must ask six open questions. Focus your questions mostly on the test
cases written, although you may ask questions that make reference to the
program source code. Try to ask the following types of questions (#
Examples), and add more as needed. In the following examples, the text
between square brackets tell you in which context the question makes sense
and provides instruction on how to exactly instantiate it.

# Examples
1. Can you give an example of a subtle edge case this test suite might
miss, and how you'd add a test for it? [Ask only if you think there are
untested edge cases.]

2. If I remove T, what kinds of bugs would no longer be caught? [Here T is
one of the test cases implemented by the student. Try to focus on
non-trivial test cases.]

3. Imagine a change C in the tested code. How would you update the test
suite? [Here C is a non-trivial code change that you will describe,
referring to a specific code location. Make the change non-trivial to
understand.]

Feel free to ask other types of tough questions that can help in assessing
the extent to which the student understands the tests they wrote. Feel free
also to instantiate the provided examples multiples times if they are
applicable to multiple tests and code locations. Remember, your goal is not
to determine whether the code is correct or not, but to find out whether
```

```
the student has understood what they have done. The student might have
copied the code from some external source such as Stack Overflow, or they
might have used tools such as GitHub Copilot or ChatGPT, so it is not
certain that all the code was written by them and, therefore, it is not
certain that they understand it.

The code for which you should pose the questions is as follows:

{{code}}

# Task

{{task}}

Next, write the six questions. In order to simplify the assessment of the
responses provided by the student, write also your answer to the questions.
Write the questions and answers and nothing else. Use the following
template to write the questions and answers:

1. <question>. ANSWER TO THE QUESTION: `<answer>`.

2. <question>. ANSWER TO THE QUESTION: `<answer>`.

3. <question>. ANSWER TO THE QUESTION: `<answer>`.

4. <question>. ANSWER TO THE QUESTION: `<answer>`.

5. <question>. ANSWER TO THE QUESTION: `<answer>`.

6. <question>. ANSWER TO THE QUESTION: `<answer>`.
```

Prompt for generating questions for maintenance task for Master students

```
You are a lecturer in Software Testing whose goal is to evaluate the code
of a program and test suite modified by a student to implement a specific
request described below (# Task).
You must carefully analyze the original code as well as the differences
between the code submitted by the student and the original code, and ask
tough questions to find out if they understand the new code they have
written.
You must ask six open questions. Focus your questions mostly on the test
cases written, although you may ask questions that make reference to the
program source code. Try to ask the following types of questions (#
Examples), and add more as needed. In the following examples, the text
between square brackets tell you in which context the question makes sense
and provides instruction on how to exactly instantiate it.

# Examples
1. Can you give an example of a subtle edge case this test suite might
miss, and how you'd add a test for it? [Ask only if you think there are
untested edge cases.]

2. If I remove T, what kinds of bugs would no longer be caught? [Here T is
one of the test cases implemented by the student. Try to focus on
non-trivial test cases.]
```

````
3. Imagine a change C in the tested code. How would you update the test
suite? [Here C is a non-trivial code change that you will describe,
referring to a specific code location. Make the change non-trivial to
understand.]

Feel free to ask other types of tough questions that can help in assessing
the extent to which the student understands the tests they wrote. Feel free
also to instantiate the provided examples multiples times if they are
applicable to multiple tests and code locations. Remember, your goal is not
to determine whether the code is correct or not, but to find out whether
the student has understood what they have done. The student might have
copied the code from some external source such as Stack Overflow, or they
might have used tools such as GitHub Copilot or ChatGPT, so it is not
certain that all the code was written by them and, therefore, it is not
certain that they understand it.

The original code of the program and test suite is as follows:

{{code}}

And the diff between the code written by the student and the original code
is as follows:

```
{{diff}}
```

# Task

{{task}}

Next, write the six questions. In order to simplify the assessment of the
responses provided by the student, write also your answer to the questions.
Write the questions and answers and nothing else. Use the following
template to write the questions and answers:

1. <question>. ANSWER TO THE QUESTION: `<answer>`.

2. <question>. ANSWER TO THE QUESTION: `<answer>`.

3. <question>. ANSWER TO THE QUESTION: `<answer>`.

4. <question>. ANSWER TO THE QUESTION: `<answer>`.

5. <question>. ANSWER TO THE QUESTION: `<answer>`.

6. <question>. ANSWER TO THE QUESTION: `<answer>`.
````

Prompt for generating questions for development task for Italian students

```
Sei un docente di Informatica il cui obiettivo è valutare il codice
sorgente di un programma scritto da uno studente per implementare una
specifica richiesta, come descritto di seguito (# Task).
Devi analizzare attentamente il codice dello studente e porre domande per
scoprire se comprende il codice che ha scritto.
Devi porre sei domande la cui risposta deve essere una riga di codice
```

pertinente alla domanda. Ad esempio, puoi chiedere dove nel codice viene verificata una certa condizione, dove viene realizzata una certa funzionalità o dove viene gestito un determinato comportamento, tra le altre opzioni.
Evita di fornire dettagli espliciti nelle domande, come stringhe letterali o condizioni, poiché in tal caso sarebbe facile individuarle nel codice senza richiedere una reale comprensione dello stesso. Evita di fare riferimento a domande precedenti che hai scritto, poiché le domande verranno poste in un ordine casuale.
Devi inoltre porre tre domande aperte, a cui lo studente dovrà rispondere con un testo. Fai domande sulla logica implementata, per verificare che lo studente abbia compreso ciò che ha realizzato.
Una di queste tre domande deve avere la seguente forma: "Descrivi in linguaggio naturale tutti i blocchi di codice presenti nella funzione <nome_funzione>". Scegli come <nome_funzione> il nome della funzione che ritieni più complessa tra quelle implementate dallo studente.
Ricorda, il tuo obiettivo non è determinare se il codice sia corretto o meno, ma scoprire se lo studente ha compreso ciò che ha fatto. È possibile che lo studente abbia copiato il codice da una fonte esterna come Stack Overflow, oppure che abbia utilizzato strumenti come GitHub Copilot o ChatGPT. Quindi non è certo che tutto il codice sia stato scritto da lui e, di conseguenza, non è certo che lui lo comprenda.

Il codice per il quale devi porre le domande è il seguente:

{{code}}

Task

{{task}}

Successivamente, scrivi le nove domande. Per automatizzare la valutazione delle risposte fornite dallo studente, scrivi anche la risposta per le domande che richiedono una riga di codice. Scrivi le domande e le risposte e nient'altro. Usa il seguente template per scrivere le domande e le risposte:

1. Copia e incolla una o più righe di codice: <domanda>. RISPOSTA ALLA DOMANDA: `<risposta>`.

2. Copia e incolla una o più righe di codice: <domanda>. RISPOSTA ALLA DOMANDA: `<risposta>`.

3. Copia e incolla una o più righe di codice: <domanda>. RISPOSTA ALLA DOMANDA: `<risposta>`.

4. Copia e incolla una o più righe di codice: <domanda>. RISPOSTA ALLA DOMANDA: `<risposta>`.

5. Copia e incolla una o più righe di codice: <domanda>. RISPOSTA ALLA DOMANDA: `<risposta>`.

6. Copia e incolla una o più righe di codice: <domanda>. RISPOSTA ALLA DOMANDA: `<risposta>`.

7. Fornisci una risposta testuale: Descrivi in linguaggio naturale tutti i blocchi di codice presenti nella funzione <nome_funzione>.

8. Fornisci una risposta testuale: <domanda>.

```
9. Fornisci una risposta testuale: <domanda>.
```

Prompt for generating questions for maintenance task for Italian students

```
Sei un docente di Informatica il cui obiettivo è valutare il codice
sorgente di un programma modificato da uno studente per implementare la
richiesta specifica descritta di seguito (# Task).
Devi analizzare attentamente il codice originale, nonché le differenze tra
il codice inviato dallo studente e quello originale, e porre domande per
scoprire se lo studente comprende il nuovo codice che ha scritto.
Devi porre sei domande la cui risposta deve essere una riga di codice
pertinente alla domanda. Ad esempio, puoi chiedere dove nel codice viene
verificata una certa condizione, dove viene realizzata una certa
funzionalità o dove viene gestito un determinato comportamento, tra le
altre opzioni.
Evita di fornire dettagli espliciti nelle domande, come stringhe letterali
o condizioni, poiché in tal caso sarebbe facile individuarle nel codice
senza richiedere una reale comprensione dello stesso. Evita di fare
riferimento a domande precedenti che hai scritto, poiché le domande
verranno poste in un ordine casuale.
Devi inoltre porre tre domande aperte, a cui lo studente dovrà rispondere
con un testo. Fai domande sulla logica implementata, per verificare che lo
studente abbia compreso ciò che ha realizzato.
Una di queste tre domande deve avere la seguente forma: "Descrivi in
linguaggio naturale tutti i blocchi di codice presenti nella funzione
<nome_funzione>". Scegli come <nome_funzione> il nome della funzione che
ritieni più complessa tra quelle implementate dallo studente.
Ricorda, il tuo obiettivo non è determinare se il codice sia corretto o
meno, ma scoprire se lo studente ha compreso ciò che ha fatto. È possibile
che lo studente abbia copiato il codice da una fonte esterna come Stack
Overflow, oppure che abbia utilizzato strumenti come GitHub Copilot o
ChatGPT. Quindi non è certo che tutto il codice sia stato scritto da lui e,
di conseguenza, non è certo che lui lo comprenda.

Il codice del programma originale è il seguente:

{{code}}

E il diff del codice scritto dallo studente e il codice originale è il
seguente:

```
{{diff}}
```

Task

{{task}}

Successivamente, scrivi le nove domande. Per automatizzare la valutazione
delle risposte fornite dallo studente, scrivi anche la risposta per le
domande che richiedono una riga di codice. Scrivi le domande e le risposte
e nient'altro. Usa il seguente template per scrivere le domande e le
risposte:

1. Copia e incolla una o più righe di codice: <domanda>. RISPOSTA ALLA
```

```
DOMANDA: `<risposta>`.

2. Copia e incolla una o più righe di codice: <domanda>. RISPOSTA ALLA
DOMANDA: `<risposta>`.

3. Copia e incolla una o più righe di codice: <domanda>. RISPOSTA ALLA
DOMANDA: `<risposta>`.

4. Copia e incolla una o più righe di codice: <domanda>. RISPOSTA ALLA
DOMANDA: `<risposta>`.

5. Copia e incolla una o più righe di codice: <domanda>. RISPOSTA ALLA
DOMANDA: `<risposta>`.

6. Copia e incolla una o più righe di codice: <domanda>. RISPOSTA ALLA
DOMANDA: `<risposta>`.

7. Fornisci una risposta testuale: Descrivi in linguaggio naturale tutti i
blocchi di codice presenti nella funzione <nome_funzione>.

8. Fornisci una risposta testuale: <domanda>.

9. Fornisci una risposta testuale: <domanda>.
```

Prompt for translating questions

```
Translate the following technical questions into {{language}}. Output the
translation and nothing else:

{{questions}}
```

Pipeline architecture

Figure 1 illustrates the overall architecture of our approach. Our implementation comprises two main components: a server with a pre-configured environment where participants could carry out the experiment, and an intelligence API in charge of generating the questions based on the code submitted by the participants. Both services are orchestrated with Docker, to facilitate deployment in any arbitrary environment.

To log in to the server, we provided participants with a unique user and password. The server was equipped with a pre-configured version of Visual Studio Code (and required plugins) so that participants could connect to it via the Remote Development plugin of VS Code, by creating an SSH connection with the provided user and password. This avoided environment issues related to language versions and incompatible libraries. All code developed by participants was stored in this server for later analysis.

Once participants finished the task, the code was sent to the intelligence API. This API was deployed in a different container in order to separate concerns (participants workspace vs interaction with external services) as well as to store OpenAI API keys and execution logs securely, inaccessible to participants. The intelligence API receives the code submitted by participants and sends it to the **o1** LLM together with one of the previously presented manually-crafted prompts, in order to generate questions related to the submitted code. The

response is automatically parsed and trimmed to contain only the questions (e.g., removing any text before or after) and, optionally, the questions are sent to the **`gpt-4o-mini`** LLM in order to translate them into a different language (e.g., if the participant declared themselves as non-proficient in English). Finally, the list of questions is sent back to the participant and saved in a TXT file where they must answer. At the end of the process, such a TXT file containing the questions and answers is stored in the server and available for us to analyze. While it is possible that participants modified the questions in the TXT file, we would detect this issue since the questions generated by OpenAI APIs are logged and stored in the intelligence API.

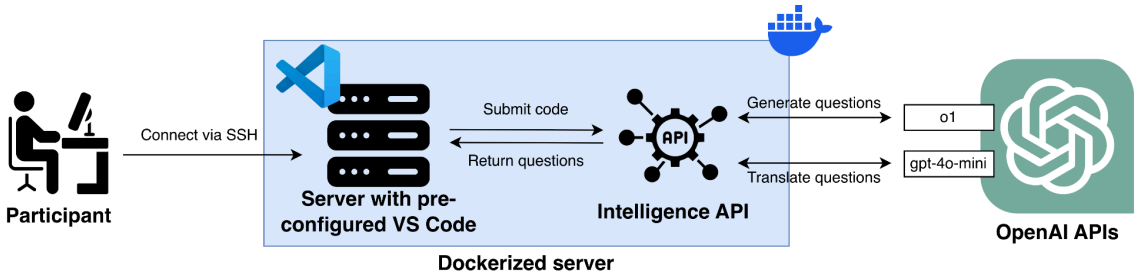

Figure 1. Pipeline architecture.

Human-in-the-loop details and reliability

Our study does not involve continued interactions between the human and the LLM, as it is a one-shot request-response pattern: the participant submits the implemented code and the LLM returns a set of questions based on such code. Moreover, the participant does not interact directly with the LLM, but rather through a proxy API which, based on the raw code submitted, crafts a complex prompt, sends it to the LLM, and parses the response generated by the LLM containing the questions. Therefore, the human-in-the-loop interaction pattern is limited.

We may clarify, however, that human involvement was crucial for two purposes: (i) to improve the quality of the questions generated by the selected LLMs *before* the execution of the experiments, and (ii) to check the quality and validity of the questions generated by the selected LLMs *after* the execution of the experiments.

Regarding the first point, we followed an iterative process to refine the prompts used to send to the LLMs to generate the questions. This process involved setting up the environment (including the same tasks to be implemented by participants), testing the complete infrastructure and evaluating the validity of the questions generated based on the prompts developed so far. We performed several rounds of this process, evolving the prompts in each of them, until finding the proper prompts to ensure the generation of high-quality non-trivial meaningful questions. Human evaluation in this process helped us improve the prompts in the following ways:

- To provide guidance on the kind of questions to generate: `you may ask where in the code a certain condition is checked, where certain functionality is accomplished, or where certain behavior is handled, among other options`.

- To avoid asking trivial questions: `Avoid providing explicit details in the questions such as literal strings or conditions, since then those would be easy to spot in the code, without actually requiring an understanding of it`.
- To target questions towards the understanding of the participant, not the correctness of the code: `Remember, your goal is not to determine whether the code is correct or not, but to find out whether the student has understood what they have done`.
- To avoid hallucinations about unimplemented code, by forcing the LLM to provide a concrete existing answer to each closed question: `In order to automate the assessment of the responses provided by the student, write also the answer to the questions asking for a line of code`.

These are some examples of improvements that we achieved in the prompts thanks to the human-in-the-loop process evaluating the questions generated iteratively.

Regarding the checking of the questions *after* we carried out the experiments, this was done to ensure that non-meaningful questions (which could still be generated by the LLM) were excluded from the evaluation. Some examples include:

- *In what way might you adapt your current data structure to handle multiple users or sessions simultaneously?* This question was discarded since it asks about how to implement an unrequested change, thus not helping in evaluating the understanding of the code written by the participant.
- *Which single line prints an error message when the input is outside the allowed range in that function?* This question was discarded because it hints about a single line and a simple textual search for "`System.err.println`" returns the answer.
- *In the CSV parameterized test for searching by exact title, why did you select just those three titles, and could you expand this coverage?* This question was discarded because it is irrelevant to evaluate the understanding of the participant of the code snippet it refers to.

Overall, manual validation and human involvement was critical to ensure the reliability of our results.

Open-model spot checks

While we used closed models in our experiments (**`o1`** and **`gpt-4o-mini`** by OpenAI), we also explored the possibility of using open-source models, which could be deployed locally and make the execution of the experiments potentially cheaper, provided that the required hardware is available. A systematic evaluation of models under different scenarios (code submitted by different participants) is out of scope, nevertheless, we selected a representative scenario and, based on it, evaluated the questions generated by four different LLMs of four different sizes and from four different providers.

Representative scenario

Below is the representative scenario chosen for this exploratory study (code submitted by one participant of our study).

main.py:

```
from util import print_menu, choose_option

if __name__ == "__main__":
    while True:
        print_menu()
        choose_option()
```

model.py:

```
from enum import Enum

class Occasion(Enum):
    BIRTHDAY = "Birthday"
    CHRISTMAS = "Christmas"
    ANNIVERSARY = "Anniversary"

class Status(Enum):
    PURCHASED = "Purchased"
    PLANNED = "Planned"

class Gift:
    def __init__(self, id: str, recipient: str, description: str, occasion:
Occasion, price: float, status: Status):
        self._id = id
        self._recipient = recipient
        self._description = description
        self._occasion = occasion
        self._price = price
        self._status = status

    @property
    def id(self):
        return self._id

    @property
    def recipient(self):
        return self._recipient

    @property
    def description(self):
        return self._description

    @property
    def occasion(self):
        return self._occasion

    @property
    def price(self):
        return self._price
```

```
    @property
    def status(self):
        return self._status

    @status.setter
    def status(self, status: Status):
        self._status = status

    def to_string(self):
        return f"ID: {self.id}\nDescription: {self.description}\nOccasion:
{self.occasion.value}\nPrice: {self.price}\nStatus: {self.status.value}"


class Option:
    def __init__(self, id: str, description: str, callable: callable,
**args: dict):
        self.id = id
        self.description = description
        self.callable = callable
        self.args = args

    def run(self):
        self.callable(**self.args)

    def to_string(self):
        return f"Description: {self.description}"
```

util.py:

```
from model import Option
from logic import add_gift, list_gifts, update_gift_status, delete_gift,
save_and_exit, empty

OPTIONS = [
    Option("1", "Add a new gift idea", add_gift),
    Option("2", "List all gift ideas", list_gifts),
    Option("3", "Update a gift status", update_gift_status),
    Option("4", "Delete a gift idea", delete_gift),
    Option("5", "Search the gifts matching a given price range", empty),
    Option("6", "Exit and save", save_and_exit),
]

def print_menu():
    print("================== MENU ==================")
    print("1. Add a new gift idea")
    print("2. List all gift ideas")
    print("3. Update a gift status")
    print("4. Delete a gift idea")
    print("5. Search the gifts matching a given price range")
    print("6. Exit and save")
    print("==========================================")
    print()
    print("Select an option: ")
    print()


def choose_option() -> int:
```

```
    option = input()
    possible_option_ids = [option.id for option in OPTIONS]
    if option not in possible_option_ids:
        print("Invalid option. Please try again.")
        choose_option()

    chosed_option = [opt for opt in OPTIONS if opt.id == option][0]
    chosed_option.run()
```

```
logic.py:
```

```
from model import Gift, Status, Occasion
from uuid import uuid4
from time import sleep
import os, pandas as pd


def load_gifts() -> list:
    if not os.path.exists("wishlist.csv"):
        return list()

    df = pd.read_csv("wishlist.csv")
    return [Gift(
                id=row["id"],
                recipient=row["recipient"],
                description=row["description"],
                price=row["price"],
                occasion=Occasion[row["occasion"].upper()],
                status=Status[row["status"].upper()]
                ) for _, row in df.iterrows()]

gifts = load_gifts()

def empty():
    print("To implement")

def add_gift():
    print("Enter the recipient's name: ")
    recipient = input()
    print("Enter the gift description: ")
    description = input()

    while True:
        try:
            print("Enter the gift price: ")
            price = float(input())
            break
        except ValueError:
            print("Please enter a valid number")

    while True:
        print("Enter the occasion: ")
        occasion = input()
        try:
            occasion = Occasion[occasion]
            break
        except:
```

```
            print("Please enter a valid occasion. Possible values are: ",
[o.name for o in Occasion])

    uuid = uuid4()
    status = Status.PLANNED
    gift =  Gift(uuid, recipient, description, occasion, price, status)
    gifts.append(gift)

    print("Gift added successfully")
    sleep(3)
    os.system("clear")


def list_gifts():
    global gifts
    os.system("clear")

    print("Here is the list of gifts: ")

    recipient_gifts = {}
    for gift in gifts:
        if gift.recipient not in recipient_gifts:
            recipient_gifts[gift.recipient] = []
        recipient_gifts[gift.recipient].append(gift)

    for recipient, gifts in recipient_gifts.items():
        print(f"\n\n## Recipient: {recipient}")
        for idx, gift in enumerate(gifts):
            print(f"-- Gift {idx+1}: -- ")
            print(gift.to_string())

    print("\n\n")

def update_gift_status():
    global gifts
    print("Enter the gift id to update: ")
    gift_id = input()
    gift = next((g for g in gifts if g.id == gift_id), None)
    if gift:
        try:
            if gift.status == Status.PURCHASED:
                gift.status = Status.PLANNED
            else:
                gift.status = Status.PURCHASED

            print("Gift status updated successfully")
            sleep(3)
            os.system("clear")
        except:
            print("Please enter a valid status. Possible values are: ",
[s.name for s in Status])
    else:
        print("Gift not found")

def delete_gift():
    global gifts
    print("Enter the gift id to delete: ")
    gift_id = input()
    gift = next((g for g in gifts if g.id == gift_id), None)
    if gift:
```

```
        gifts.remove(gift)
        print("Gift deleted successfully")
    else:
        print("Gift not found")

def save_and_exit():
    global gifts
    df = pd.DataFrame()
    for gift in gifts:
        gift_data = {
            "id": [gift.id],
            "recipient": [gift.recipient],
            "description": [gift.description],
            "price": [gift.price],
            "occasion": [gift.occasion.value],
            "status": [gift.status.value]
        }
        gift_df = pd.DataFrame(gift_data)
        df = pd.concat([df, gift_df], ignore_index=True)

    if df.empty:
        os.remove("wishlist.csv")
    else:
        df.to_csv("wishlist.csv", index=False)

    print("Gifts saved successfully")
    exit()
```

Based on this code, we crafted the prompt previously presented (“Prompt for generating questions for development task”) and sent it to four LLMs. The questions generated by the different LLMs are shown below. For each model, we specify name and size, and detail whether the questions generated are meaningful or not.

DeepSeek R1 (685B)

Most questions generated by this LLM are meaningful. Exceptionally, questions #4 and #6 are relatively trivial, as they hint at very specific parts of the code (a setter, PLANNED and PURCHASED values) that would be easy to spot with a simple search.

```
1. Copy and paste a line or lines of code: Where are the menu options
stored and initialized?

2. Copy and paste a line or lines of code: Where is the unique identifier
for new gifts generated?

3. Copy and paste a line or lines of code: Where is input validation
implemented for numeric values?

4. Copy and paste a line or lines of code: Where is the status property
modified through a setter?

5. Copy and paste a line or lines of code: Where are gifts grouped by
recipient for display purposes?

6. Copy and paste a line or lines of code: Where is the toggle between
```

```
PLANNED and PURCHASED statuses implemented?

7. Provide a textual answer: Describe in natural language all code blocks
in the save_and_exit function.

8. Provide a textual answer: Why does the search functionality (option 5)
use the empty function instead of a proper implementation?

9. Provide a textual answer: What potential issue could arise from the
recursive call to choose_option() when handling invalid input?
```

Qwen3 Coder (480B)

Also in this case most questions are meaningful, with the only exception of question #3, for the same reason as in the previous LLM (question #6).

```
1. Copy and paste a line or lines of code: Where is the unique ID generated
when adding a new gift?

2. Copy and paste a line or lines of code: Where does the application load
the existing gifts from the CSV file when starting up?

3. Copy and paste a line or lines of code: Where is the status of a gift
toggled between Purchased and Planned?

4. Copy and paste a line or lines of code: Where is the validation
performed to check if the user input matches one of the available menu
options?

5. Copy and paste a line or lines of code: Where is the exception handling
implemented to ensure a valid numeric price is entered?

6. Copy and paste a line or lines of code: Where is the CSV file created or
updated when saving the gift list before exiting?

7. Provide a textual answer: Describe in natural language all code blocks
in the list_gifts function.

8. Provide a textual answer: Explain how the application handles the case
when the wishlist CSV file does not exist during the loading process.

9. Provide a textual answer: What is the purpose of using the global
keyword in functions like update_gift_status and list_gifts?
```

Llama 3.1 (70B)

In this case, while all questions are reasonable, we see a pattern of the LLM hinting at the specific function where to look at for the asked content (questions #1, #3, #5 and #6). We emphasize that we tailored our prompts to avoid this behavior with the specific LLM used in our study (**o1** model by OpenAI), and different LLMs may require different tailored prompts. It is also possible that smaller models (Llama 3.1 is about 20 times smaller than DeepSeek R1) may be unable to follow long and complex instructions like the ones provided in our prompt. This issue is particularly notable in the last open-source LLM that we tested, the smallest one, shown in the next section.

```
1. Copy and paste a line or lines of code: Where the gift status is updated
in the `update_gift_status` function.

2. Copy and paste a line or lines of code: Where the gifts are loaded from
the CSV file when the application starts.

3. Copy and paste a line or lines of code: Where the gift is added to the
list of gifts in the `add_gift` function.

4. Copy and paste a line or lines of code: Where the gifts are saved to the
CSV file before exiting the application.

5. Copy and paste a line or lines of code: Where the gift is removed from
the list of gifts in the `delete_gift` function.

6. Copy and paste a line or lines of code: Where the unique ID is generated
for a new gift in the `add_gift` function.

7. Provide a textual answer: Describe in natural language all code blocks
in the `list_gifts` function.

8. Provide a textual answer: What is the purpose of the `recipient_gifts`
dictionary in the `list_gifts` function, and how is it used to display the
gifts?

9. Provide a textual answer: How does the application ensure data
persistence between sessions, and what file format is used to store the
wishlist?
```

Gemma 3 (4B)

In this case, the LLM was totally unable to craft a proper list of questions, probably due to its small size, which limits its ability to process long and complex requests (containing instructions, code, required output format, etc.).

```
1. Copy and paste a line of code: `print_menu()`.

2. Copy and paste a line of code: `choose_option()`.

3. Copy and paste a line of code: `gift = Gift(uuid, recipient,
description, occasion, price, status)`.

4. Copy and paste a line of code: `df.to_csv("wishlist.csv", index=False)`.

5. Copy and paste a line of code: `os.system("clear")`.

6. Copy and paste a line of code: `if option not in possible_option_ids:`.

7. Provide a textual answer: Describe in natural language all code blocks
in the `add_gift` function.

8. Copy and paste a line of code: `gift.status = Status.PURCHASED`.

9. Copy and paste a line of code: `if gift.status == Status.PURCHASED:`.
```

All in all, we acknowledge that our study could be carried out with open-source models, provided that prompts are carefully tailored for those models, following a human-in-the-loop process as detailed in previous sections.

Non-determinism and model drift

Contrary to other experiments involving LLMs, the validity of our results is not affected by their inherent non-determinism. This is because our results are not related to the performance of the LLMs used in some particular task, but rather to the conclusions derived from the experiments with human participants. Furthermore, all outputs by all LLMs (i.e., questions generated) were double-checked by two independent authors, discarding those deemed as inadequate. In our experiments, we used LLMs as a tool to realize our goal: automatically asking questions about certain code. Therefore, non-determinism was more encouraged than avoided, since we aimed at generating diverse and creative questions.

Data drift is not a concern either since we used an up-to-date model (`o1`) to process the code submitted by participants. Participants used Python version 3.10 and Java version 17 to carry out the required coding tasks, and the selected model is proficient in both language versions. Should the experiment be replicated in the future with newer languages or language versions, avoiding data drift would be as simple as using models that have been trained with such languages.

Ethical considerations and data privacy

All participants provided informed consent prior to the study via an initial form shared with them before the start of the experiment. Since participation was entirely voluntary and based on informed consent, formal IRB or ethics committee approval was not required. To ensure data privacy, no personal details or identifying information about the participants left institutional boundaries; only the source code authored during the tasks was shared with external LLMs. Regarding the use of the OpenAI API, we adhered to their standard data retention policy, which states that content submitted via the completions API may be retained for up to 30 days for safety and abuse monitoring purposes, after which it is deleted. Furthermore, data sent over the API is not used for model training unless explicit agreement is provided, which was not the case for this study. No additional data-processing agreements were established beyond these standard terms.